Security Baseline and Controlled Testing

Windows Endpoint Baseline, Linux Access Control, Network Traffic Analysis, and Controlled Security Testing in an Isolated Virtual Lab.

1. OBJECTIVE

This lab establishes normal operating baselines for a Windows endpoint and a Linux host, applies role-based access control on Linux, captures and interprets routine network traffic, and then performs controlled reconnaissance and authentication testing to observe how each system logs that activity. The goal is to compare expected, everyday behaviour against the evidence generated by deliberate, authorised testing, and to draw practical security conclusions from the difference.

2. LAB ENVIRONMENT

The exercise was carried out in an isolated virtual network using the following machines:

Role	Platform	Purpose
Windows Endpoint	Windows 11 Pro	Baseline profiling such as system info, firewall, services, ports, processes
Linux Host (Kali)	Kali Linux	User/group administration, access control testing, SSH logging
Target Host	Ubuntu	Network traffic analysis and controlled scan/brute-force target

Note: private (internal) IP addresses are used throughout this lab and are not sensitive outside the lab network, any addresses visible in screenshots have been masked where practical.

3. WINDOWS ENDPOINT BASELINE

The Windows endpoint was profiled to document its normal configuration before any testing, covering host firewall policy, running services, listening network ports, and process relationships.

3.1 Host Firewall Policy

The built-in firewall was queried across all three profiles (Domain, Private, Public). All three profiles were active ("State: ON") with a default policy of Block Inbound / Allow Outbound, confirming the endpoint rejects unsolicited inbound connections by default.

```
C:\Users\user>netsh advfirewall show allprofiles

Domain Profile Settings:
----------------------------------------------------------------------
State                                 ON
Firewall Policy                       BlockInbound,AllowOutbound
LocalFirewallRules                    N/A (GPO-store only)
LocalConSecRules                      N/A (GPO-store only)
InboundUserNotification               Enable
RemoteManagement                      Disable
UnicastResponseToMulticast            Enable

Logging:
LogAllowedConnections                 Disable
LogDroppedConnections                 Disable
FileName                              %systemroot%\system32\LogFiles\Firewall\pfirewall.log
MaxFileSize                           4096


Private Profile Settings:
----------------------------------------------------------------------
State                                 ON
Firewall Policy                       BlockInbound,AllowOutbound
LocalFirewallRules                    N/A (GPO-store only)
LocalConSecRules                      N/A (GPO-store only)
InboundUserNotification               Enable
RemoteManagement                      Disable
UnicastResponseToMulticast            Enable

Logging:
LogAllowedConnections                 Disable
LogDroppedConnections                 Disable
FileName                              %systemroot%\system32\LogFiles\Firewall\pfirewall.log
MaxFileSize                           4096


Public Profile Settings:
----------------------------------------------------------------------
State                                 ON
Firewall Policy                       BlockInbound,AllowOutbound
LocalFirewallRules                    N/A (GPO-store only)
LocalConSecRules                      N/A (GPO-store only)
InboundUserNotification               Enable
RemoteManagement                      Disable
UnicastResponseToMulticast            Enable

Logging:
LogAllowedConnections                 Disable
LogDroppedConnections                 Disable
FileName                              %systemroot%\system32\LogFiles\Firewall\pfirewall.log
MaxFileSize                           4096

Ok.
```

Figure 1 - Windows Defender Firewall profile settings (netsh advfirewall show allprofiles)

3.2 Running Services

Active services were listed and reviewed for anything unexpected. All observed services (Windows Audio, DHCP Client, Cryptographic Services, Background Tasks Infrastructure, and similar) are standard Windows components, consistent with a normal, unmodified endpoint.

```
PS C:\Users\user> Get-Service | Where-Object {$_.Status -eq "Running"}

Status   Name               DisplayName
------   ----               -----------
Running  AdobeARMservice    Adobe Acrobat Update Service
Running  AnyDesk            AnyDesk Service
Running  Appinfo            Application Information
Running  AppXSvc            AppX Deployment Service (AppXSVC)
Running  AudioEndpointBu... Windows Audio Endpoint Builder
Running  Audiosrv           Windows Audio
Running  BFE                Base Filtering Engine
Running  BrokerInfrastru... Background Tasks Infrastructure Ser...
Running  bthserv            Bluetooth Support Service
Running  camsvc             Capability Access Manager Service
Running  cbdhsvc_a5806      Clipboard User Service_a5806
Running  CDPSvc             Connected Devices Platform Service
Running  CDPUserSvc_a5806   Connected Devices Platform User Ser...
Running  ClipSVC            Client License Service (ClipSVC)
Running  CoreMessagingRe... CoreMessaging
Running  cphs               Intel(R) Content Protection HECI Se...
Running  cplspcon           Intel(R) Content Protection HDCP Se...
Running  CryptSvc           Cryptographic Services
Running  DcomLaunch         DCOM Server Process Launcher
Running  DeviceAssociati... Device Association Service
Running  DeviceInstall      Device Install Service
Running  DevicesFlowUser... DevicesFlow_a5806
Running  DevQueryBroker     DevQuery Background Discovery Broker
Running  Dhcp               DHCP Client
Running  DiagTrack          Connected User Experiences and Tele...
```

Figure 2 - Running services (Get-Service | Where-Object Status -eq "Running")

3.3 Listening Network Ports

Listening TCP ports were enumerated to identify which services accept inbound connections. Ports were bound to 0.0.0.0 / [::] (all interfaces) or loopback, consistent with standard Windows services (RPC, SMB, and local application ports) rather than an exposed, internet-facing service.

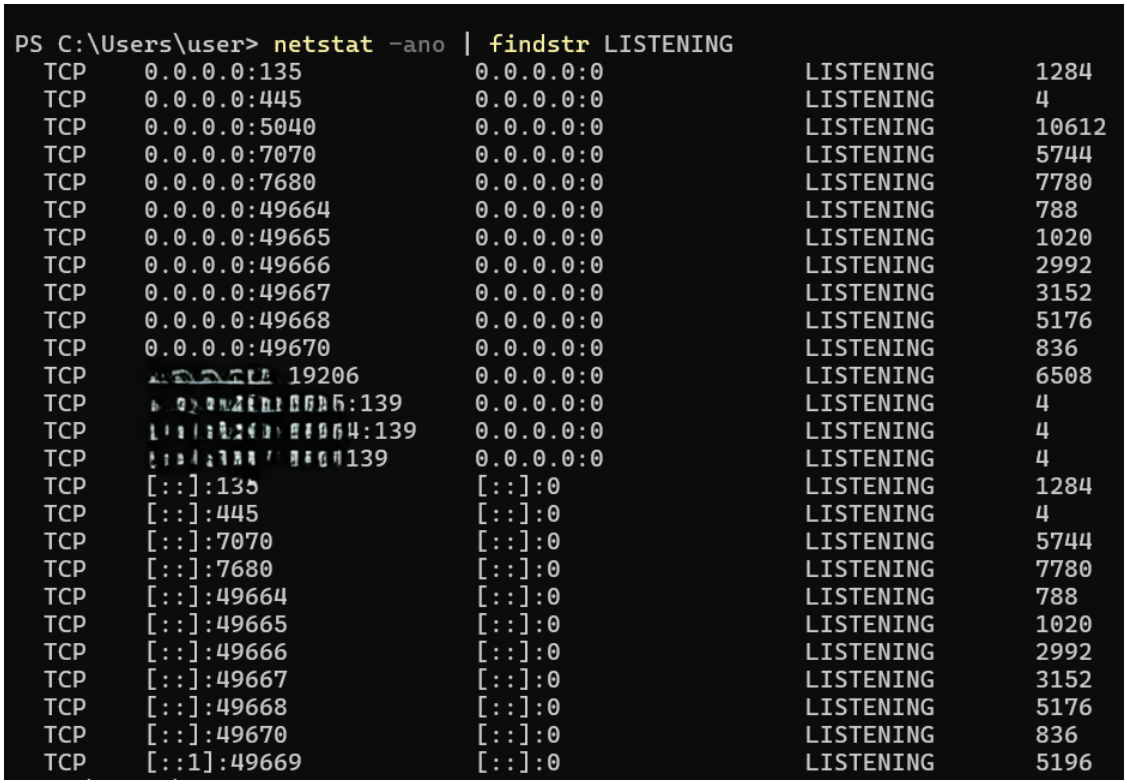


Figure 3 - Listening TCP ports (netstat -ano | findstr LISTENING)

3.4 Process and Parent-Child Relationships

Core system processes (smss.exe, csrss.exe, services.exe, dllhost.exe, RuntimeBroker.exe) were checked against their expected parent process IDs and executable paths. Each process resolved to its expected Windows system path, with no unexpected parent-child relationships. This is a key indicator used to rule out process-impersonation techniques.

```
PS C:\Users\user> Get-CimInstance Win32_Process |
>>   Where-Object { $_.Name -match "smss|csrss|services|dllhost|RuntimeB
roker|brave" } |
>>   Select Name, ProcessId, ParentProcessId, ExecutablePath

Name                    ProcessId ParentProcessId ExecutablePath
----                    --------- --------------- --------------
smss.exe                      644               4
csrss.exe                     936             820
services.exe                  836            1020
BraveCrashHandler.exe       10904            9792
BraveCrashHandler64.exe      9272            9792
csrss.exe                    5364           13980
RuntimeBroker.exe           10864            1148 C:\Windows\System32\Ru
ntimeB...
RuntimeBroker.exe            2108            1148 C:\Windows\System32\Ru
ntimeB...
dllhost.exe                  4116            1148 C:\WINDOWS\system32\Dl
lHost.exe
RuntimeBroker.exe            6768            1148 C:\Windows\System32\Ru
ntimeB...
RuntimeBroker.exe           14656            1148 C:\Windows\System32\Ru
ntimeB...
brave.exe                   13160           13180 C:\Program Files\Brave
Softwa...
brave.exe                    4416           13160 C:\Program Files\Brave
```

Figure 4 - Process tree check (Get-CimInstance Win32_Process, filtered to core system processes)

3.5 Windows Security Event Log

The Security event log was reviewed for account-management and logon activity. Event ID 5379 (Credential Manager credentials read), 4624 (successful logon) and 4672 (special/privileged logon) appeared consistently during normal use, establishing a baseline signature for routine account activity on this endpoint.

Keywords	Date and Time	Source	Event ID	Task Category
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management
Audit Success	10/8/2026 12:16:30 PM	Microsoft Windows security...	5379	User Account Management

Figure 5 - Windows Security Event Log, filtered to Account Management and Logon events

4. LINUX USER ADMINISTRATION AND ACCESS CONTROL

On the Linux host, two standard user accounts (referred to here as User A and User B) and a dedicated group were created to demonstrate role-based access control (RBAC) and discretionary access control (DAC) enforcement, then deliberately tested to confirm the controls actually held.

4.1 Account and Group Provisioning

Two user accounts were created with useradd, passwords were set, and a dedicated group was created with groupadd. Both accounts were then added to the group with usermod -aG, and getent group was used to confirm membership.

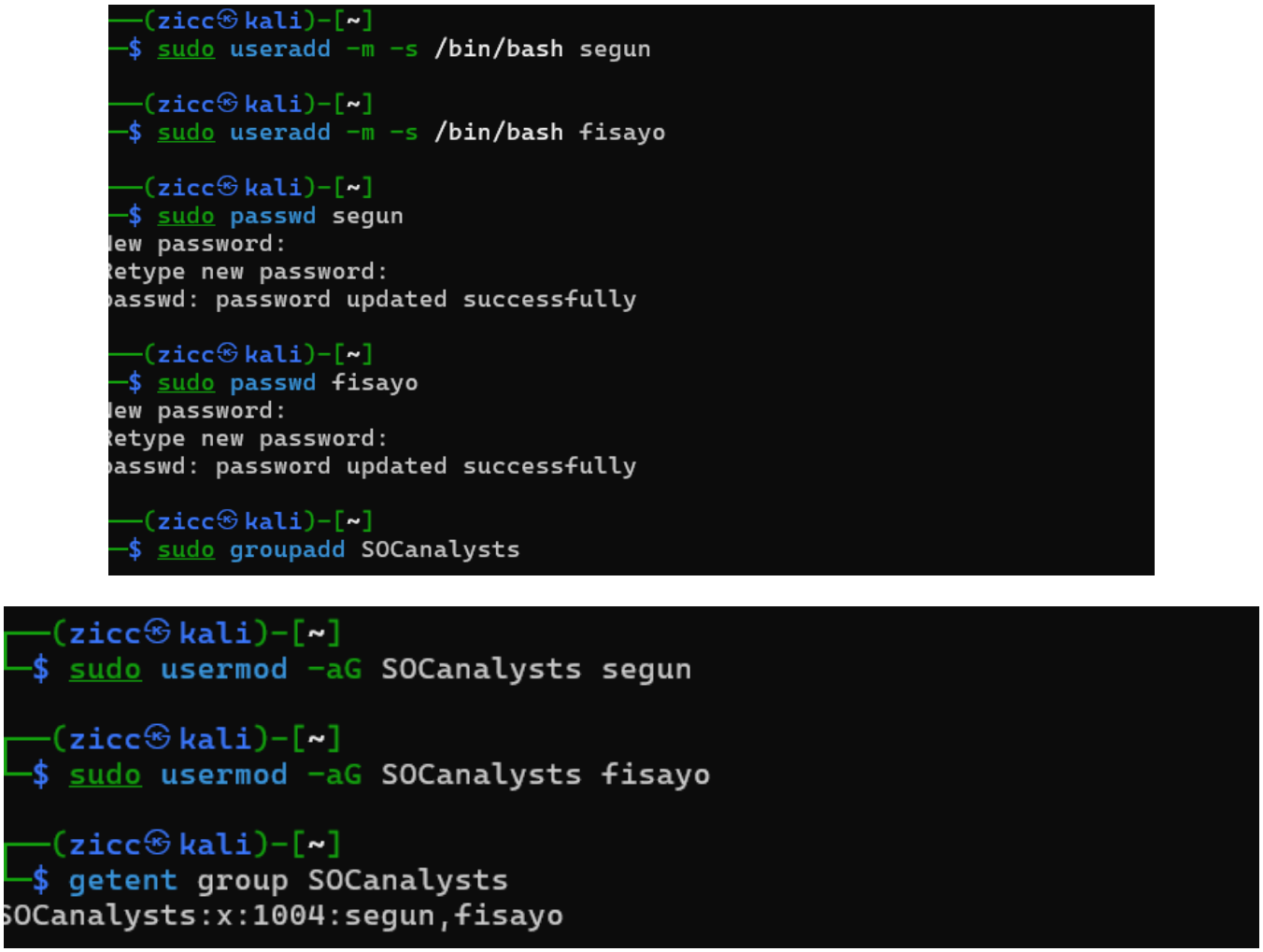


Figure 6 - User account creation, password assignment, and group creation

4.2 Directory Permission Enforcement (chmod 770)

A project directory was restricted with chmod 770 and group ownership assigned to the new group, so only the owner and group members could read or write its contents, with all other users blocked. Listing

the directory and reading its file succeeded only for the authorised user, confirming the restriction was enforced at the filesystem level rather than merely configured.

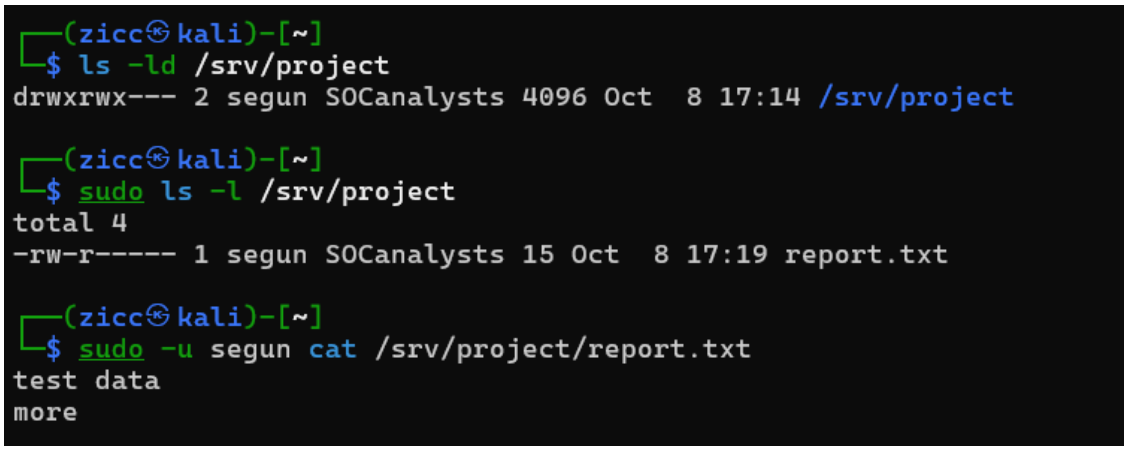


Figure 7 - Directory listing and file read succeeding for an authorised group member

4.3 Privilege Boundary Testing

To confirm the access control was not merely cosmetic, a user was removed from the authorised group and the same read attempt was repeated, which then failed with Permission denied, exactly as expected. A separate test confirmed that a non-administrative account could not run sudo commands, and was correctly rejected with "***user is not in the sudoers file***," with the attempt itself recorded in the system's authorisation log.

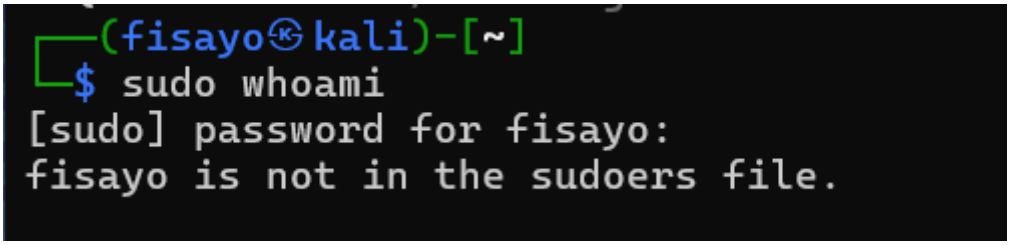


Figure 8 - Non-privileged account correctly denied sudo access

4.4 SSH Authentication Logging

SSH logins were tested for both accounts. One account authenticated successfully on the first attempt; the other was tested with incorrect credentials three times before a correct one succeeded. The system log (journalctl -u ssh) captured every attempt individually, labelling each as Failed password or Accepted password with a timestamp, demonstrating that authentication telemetry is reliable evidence for distinguishing a mistyped login from a genuine access failure.



Figure 9 - SSH authentication log showing failed attempts followed by a successful login

4.5 Login History

The last command was used to review recent login sessions across all accounts, cross-checking session start times, duration, and origin against the authentication log to confirm the two sources agreed.

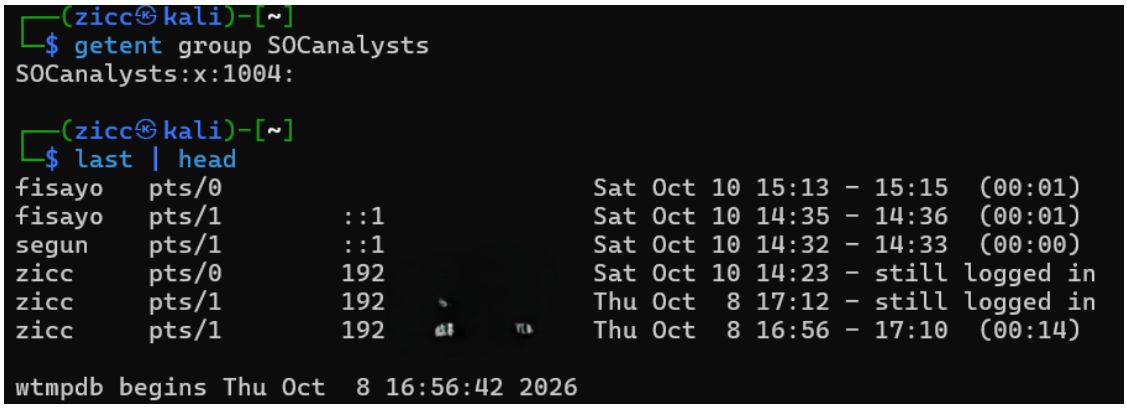


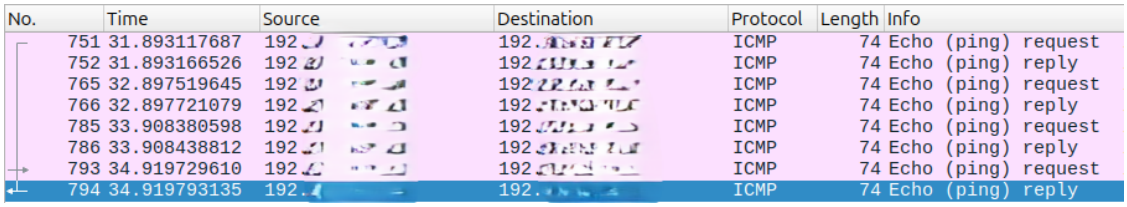
Figure 10 - Recent login history (last)

5. NETWORK TRAFFIC ANALYSIS (WIRESHARK)

With a packet capture running on the lab network, routine traffic was generated and examined to establish what normal ICMP and HTTP traffic looks like between hosts on this network, before any security testing began.

5.1 ICMP (Ping) Baseline

A standard ping exchange was captured: An Echo Request followed immediately by an Echo Reply for each sequence number, with a consistent round-trip time. This is the expected pattern for a healthy connection between two hosts on the same network.

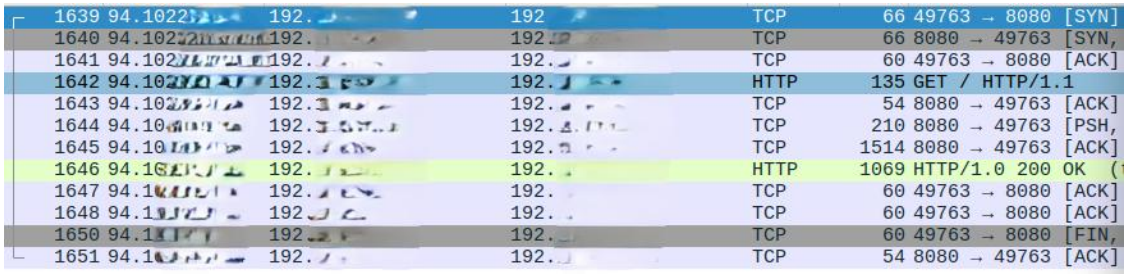


No.	Time	Source	Destination	Protocol	Length	Info
751	31.893117687	192	192	ICMP	74	Echo (ping) request
752	31.893166526	192	192	ICMP	74	Echo (ping) reply
765	32.897519645	192	192	ICMP	74	Echo (ping) request
766	32.897721079	192	192	ICMP	74	Echo (ping) reply
785	33.908380598	192	192	ICMP	74	Echo (ping) request
786	33.908438812	192	192	ICMP	74	Echo (ping) reply
793	34.919729610	192	192	ICMP	74	Echo (ping) request
794	34.919793135	192.	192.	ICMP	74	Echo (ping) reply

Figure 11 - ICMP echo request/reply pairs

5.2 HTTP Request/Response

An HTTP request to a web service on the target host was captured end-to-end: the TCP three-way handshake (SYN, SYN-ACK, ACK), followed by the GET / HTTP/1.1 request and an HTTP/1.0 200 OK response, and finally a clean connection teardown (FIN, ACK). Because HTTP is unencrypted, the full request and response are visible in plain text. This is a point worth noting when comparing it to encrypted protocols such as SSH or HTTPS.



1639	94.1022	192.	192	TCP	66	49763 → 8080 [SYN]
1640	94.102	192.	192	TCP	66	8080 → 49763 [SYN,
1641	94.102	192.	192.	TCP	60	49763 → 8080 [ACK]
1642	94.10	192.	192.	HTTP	135	GET / HTTP/1.1
1643	94.10	192.	192.	TCP	54	8080 → 49763 [ACK]
1644	94.10	192.	192.	TCP	210	8080 → 49763 [PSH,
1645	94.10	192.	192.	TCP	1514	8080 → 49763 [ACK]
1646	94.1	192.	192.	HTTP	1069	HTTP/1.0 200 OK
1647	94.1	192.	192.	TCP	60	49763 → 8080 [ACK]
1648	94.1	192	192.	TCP	60	49763 → 8080 [ACK]
1650	94.1	192	192.	TCP	60	49763 → 8080 [FIN,
1651	94.1	192.	192.	TCP	54	8080 → 49763 [ACK]

Figure 12 - Full HTTP transaction: TCP handshake, GET request, 200 OK response, and teardown

6. CONTROLLED SECURITY TESTING

With the baseline established, two controlled, authorised activities were performed against the target host to observe how the resulting telemetry differs from the traffic and logs captured above.

6.1 Port Scan (Nmap)

A TCP SYN scan was run against the target host on a small set of common ports. The scan completed in under a second and identified SSH and HTTPS as open, HTTP as closed, and an HTTP-proxy service listening on an alternate port, giving a quick, low-noise picture of exposed services without establishing full connections on each port.

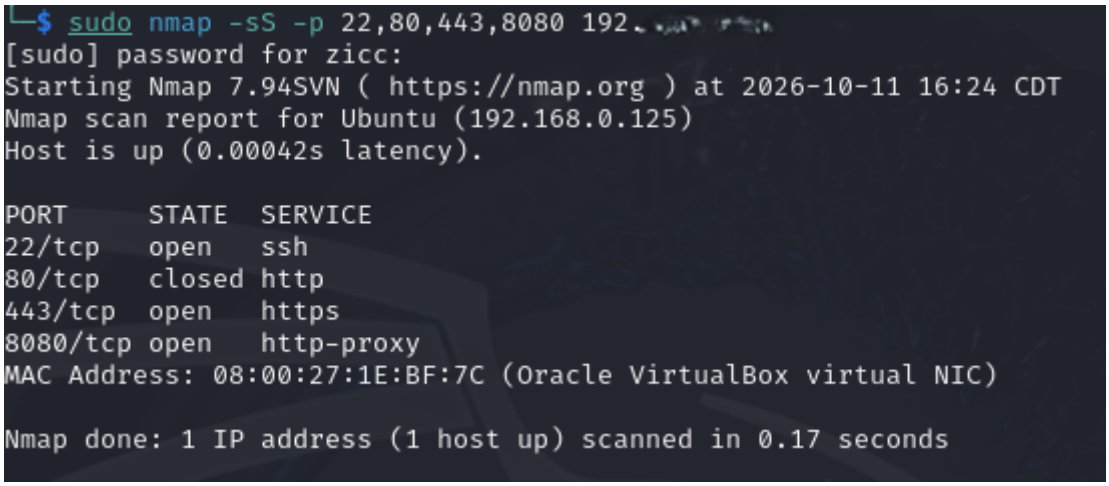


Figure 13 - Nmap SYN scan result against the target host

6.2 Controlled SSH Authentication Attempts

A series of SSH connection attempts using an invalid/incorrect account were made against the target host from the testing machine. Client-side, the terminal returned repeated "***Permission denied***" responses before the connection was dropped.

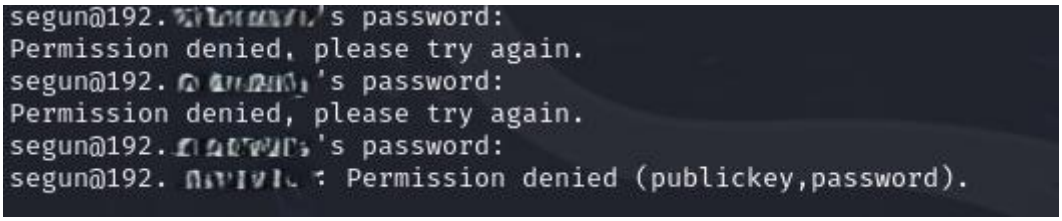


Figure 14 - Client-side view of repeated authentication failures

Server-side, the target host's authentication log recorded the same event independently: a PAM authentication failure, three Failed password entries for the invalid account, and a final "Connection closed

… [preauth]” line marking the session's termination before authentication completed. The two logs; client and server, corroborate each other and together form a complete record of the attempt.

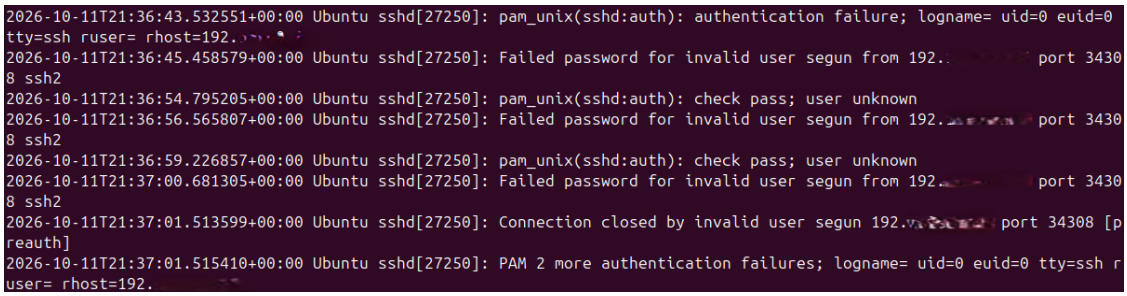


Figure 15 - Server-side authentication log for the same attempt

7. BASELINE VS. CONTROLLED ACTIVITY

Observation	Normal Baseline	Controlled Activity	Security Relevance
SSH logins	One attempt, immediately accepted.	Multiple rapid failures against one account before/without success.	A short burst of failed logins against one account is the classic signature of password guessing, it is worth investigating, though not proof of compromise on its own.
Port exposure	Not scanned during normal use.	Nmap enumerated open ports (SSH, HTTPS/proxy) in under a second.	A fast, targeted scan across multiple ports with no completed sessions is a reconnaissance signature that often precedes an attack.
Directory access	Authorised group member reads/writes freely	Same action denied once group membership was revoked	Confirms access control is enforced by the OS, not just configured on paper.
Privileged commands	N/A	sudo denied for a non-privileged account, with the attempt itself logged.	Shows that even failed privilege-escalation attempts leave an auditable trail.

An unusual pattern is a reason to investigate, not a conclusion of compromise. In this lab, every "***controlled activity***" event is confirmed benign because it was performed intentionally and is documented above. In a live environment, the same evidence would need further correlation (source attribution, account history, concurrent alerts) before reaching a conclusion.

8. FINDINGS AND RECOMMENDATIONS

Findings

- Both the Windows and Linux hosts produced clear, attributable logs for authentication and account-management events, which is the foundation of any incident investigation.

- Discretionary access controls (chmod, group membership) on Linux were correctly enforced at the OS level, not just at the configuration level.
- A short, low-noise port scan completed before any rate-limiting or blocking occurred, showing that log review alone is not sufficient to catch reconnaissance in real time.
- Repeated failed SSH logins were fully captured in the system log, but nothing on the host automatically responded to or blocked the activity.

Recommendations

- Deploy an automated intrusion-prevention tool (e.g. Fail2ban) to detect and temporarily block repeated authentication failures from the same source.
- Move toward SSH key-based authentication and disable password login where operationally feasible.
- Apply a default-deny firewall policy on exposed hosts, permitting only the ports and source addresses required.
- Forward authentication and firewall logs to a central log-management or SIEM platform rather than relying on manual, per-host log review.
- Periodically re-run this baseline-versus-activity comparison to confirm new controls are producing the expected telemetry.

9. CONCLUSION

This lab demonstrated end-to-end security monitoring fundamentals: establishing what “normal” looks like on a Windows endpoint and a Linux host, enforcing and verifying access controls, capturing routine network traffic, and then generating controlled security events to see how each layer of the environment records them. The exercise confirmed that logging across both operating systems is detailed and reliable, but also that detection and response, recognising and acting on that evidence quickly requires additional tooling beyond logging alone.

10. CHALLENGES FACED

1. Capture permissions: `tshark` run with `sudo` failed to write the capture file to the home directory because it drops root privileges after starting, requiring the output path to be changed to `/tmp` or `-Z none` used to retain privileges.

2. Wrong capture interface: Wireshark/tshark initially captured on the wrong network adapter, requiring a test ping to confirm the correct interface.

3. **DNS traffic location**: DNS queries went to the network gateway rather than the expected host, requiring the capture filter to be adjusted once the actual resolver was identified.

4. Noise in captures: unrelated background traffic cluttered early captures, requiring host-specific filters to isolate the relevant lab traffic.

5. Inaccessible reference documentation: a linked Notion page could not be scraped automatically since it requires JavaScript to render, requiring its content to be copied manually instead.

6. Public-document review: several screenshots contained identifying hostnames, usernames, or IPs, requiring a separate redaction and selection pass before they could be used in the shared report.

7.Evidence coordination: screenshots collected across multiple sessions had to be carefully matched to the correct command and section during final write-up.